

Conferencia 4: análisis amortizado

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

En la conferencia 3 usamos un potencial que solo bajaba, para demostrar que un algoritmo termina. Hoy el potencial sube y baja, y lo usamos para otra cosa: pagar las operaciones caras con lo que se ahorró en las baratas. La pregunta deja de ser "cuánto cuesta la peor operación" y pasa a ser "cuánto cuesta una secuencia de m operaciones, en el peor caso, dividido por m ". Vamos a verlo sobre tres estructuras, en orden de dificultad, y en la última el potencial ni siquiera va a ser una fórmula.

1. La peor operación miente

Un arreglo dinámico, como la lista de Python o el vector de C++, guarda sus elementos en un bloque de memoria con cierta capacidad. Cuando un append encuentra el bloque lleno, pide uno del doble de tamaño y copia todo. Ese append cuesta $\Theta(n)$. Si analizamos operación por operación, n append cuestan $O(n^2)$. Es verdadero, y es inútil, porque no pasa nunca:

```
def copias_de_n_appends(n):
    capacidad, tamaño, copias = 1, 0, 0
    for _ in range(n):
        if tamaño == capacidad:           # lleno: duplicar y copiar todo
            copias += tamaño
            capacidad *= 2
        tamaño += 1
    return copias

for n in [10, 1_000, 1_000_000]:
    c = copias_de_n_appends(n)
    print(f"n = {n:>9}, copias = {c:>9}, copias por append = {c / n:.2f}")
```

n =	10	copias =	15	copias por append = 1.50
n =	1,000	copias =	1,023	copias por append = 1.02
n =	1,000,000	copias =	1,048,575	copias por append = 1.05

Las copias se hacen cuando el tamaño es 1, 2, 4, ..., así que en total son $1 + 2 + 4 + \dots + 2^k < 2n$.

Sumando la escritura de cada elemento, n append cuestan menos de $3n$: $O(1)$ por operación, **en promedio sobre la secuencia**.

Esa es la idea, y conviene fijar el vocabulario antes de seguir.

Definición. Un **costo amortizado** para una estructura es una asignación de un número $\hat{c}_i$ a cada operación tal que, para **toda** secuencia de operaciones,

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m \hat{c}_i,$$

donde c_i es el costo real de la operación i .

Hay que decir lo que esto **no** es, porque es lo que se confunde siempre. El costo amortizado no es un caso promedio. No hay entradas aleatorias ni probabilidad en ninguna parte: la desigualdad vale para toda secuencia, incluida la peor que un adversario pueda construir. Es una garantía de peor caso, solo que sobre secuencias y no sobre operaciones sueltas. En Estructuras de Datos se enunció que append es $O(1)$ amortizado. Hoy lo demostramos, y demostramos cosas bastante más difíciles.

2. La cola con dos pilas: tres métodos

Una cola con dos pilas: se encola apilando en una pila de **entrada**, y se desencola desapilando de una pila de **salida**. Cuando la de salida está vacía, se vuelca entera la de entrada sobre ella, lo que invierte el orden y deja al más viejo arriba. Medimos el costo real como el número de operaciones de pila (push o pop).

```
class Cola:
    """Cola con dos pilas. Cada método devuelve su costo real: cuántos push y
    pop hizo sobre las pilas."""

    def __init__(self):
        self.entrada, self.salida = [], []

    def encolar(self, x):
        self.entrada.append(x)
        return 1

    def desencolar(self):
        costo = 0
        if not self.salida:                # volcar: pop de una, push en la otra
            while self.entrada:
                self.salida.append(self.entrada.pop())
                costo += 2
            self.salida.pop()
        return costo + 1

    def potencial(self):
        return 2 * len(self.entrada)
```

Un desencolar que encuentra la salida vacía y k elementos en la entrada cuesta $2k + 1$, que puede ser $\Theta(n)$. Hay tres maneras de demostrar que no importa.

Método agregado. Se acota el total directamente. Cada elemento, a lo largo de su vida, entra en la pila de entrada, sale de ella, entra en la de salida y sale de ella: cuatro operaciones como mucho. Una secuencia con m operaciones encola a lo sumo m elementos, así que cuesta a lo sumo $4m$. Es $O(1)$ amortizado.

Método de contabilidad. Se le cobra a cada operación una tarifa fija, a veces más de lo que cuesta, y el sobrante se deja como crédito guardado en la estructura para pagar operaciones futuras. Cobramos 3 por encolar y 1 por desencolar. Encolar gasta 1 en su push y deja 2 de crédito pegado al elemento. Cuando llega la mudanza, cada elemento paga su pop y su push con esos 2. El pop final lo paga la tarifa del desencolar. La demostración es el invariante "todo elemento de la pila de entrada tiene 2 de crédito", que garantiza que el crédito nunca es negativo.

Método del potencial. Se define una función Φ del estado de la estructura, y el costo amortizado de la operación i es

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1},$$

el costo real más lo que cambió el potencial. Sumando, los términos del potencial se telescopian:

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m c_i + \Phi_m - \Phi_0.$$

Así que $\sum c_i \leq \sum \hat{c}_i$ siempre que $\Phi_m \geq \Phi_0$. **Ese es el paso que todo el mundo olvida:** un potencial que puede terminar por debajo de donde empezó no demuestra nada. Lo habitual es arrancar en $\Phi_0 = 0$ y exigir $\Phi \geq 0$.

Para la cola, $\Phi = 2 \cdot |\text{entrada}|$. Un encolar cuesta 1 y sube el potencial en 2: amortizado 3. Un desencolar sin mudanza cuesta 1 y no cambia el potencial: amortizado 1. Un desencolar con mudanza de k elementos cuesta $2k + 1$ y baja el potencial en $2k$: amortizado 1. El potencial es 0 al principio y nunca es negativo.

Los tres métodos dan lo mismo, y no es casualidad: el crédito de la contabilidad, sumado sobre toda la estructura, es el potencial. El potencial es la contabilidad sin nombres, sin decir qué elemento guarda qué moneda, y por eso escala a estructuras donde ponerle nombre al crédito sería imposible. Comprobémoslo sobre una secuencia larga, con un assert que vigila el costo amortizado paso a paso.

```
random.seed(1)
q, tamaño, reales = Cola(), 0, []
for _ in range(100_000):
    antes = q.potencial()
    if tamaño == 0 or random.random() < 0.55:
        c = q.encolar(0)
        tamaño += 1
        assert c + q.potencial() - antes == 3
    else:
        c = q.desencolar()
        tamaño -= 1
        assert c + q.potencial() - antes == 1
    reales.append(c)
print(f"operaciones: {len(reales):,}   la más cara: {max(reales):,}   "
      f"costo real promedio: {sum(reales) / len(reales):.2f}")
```

operaciones: 100,000 la más cara: 18,345 costo real promedio: 2.00

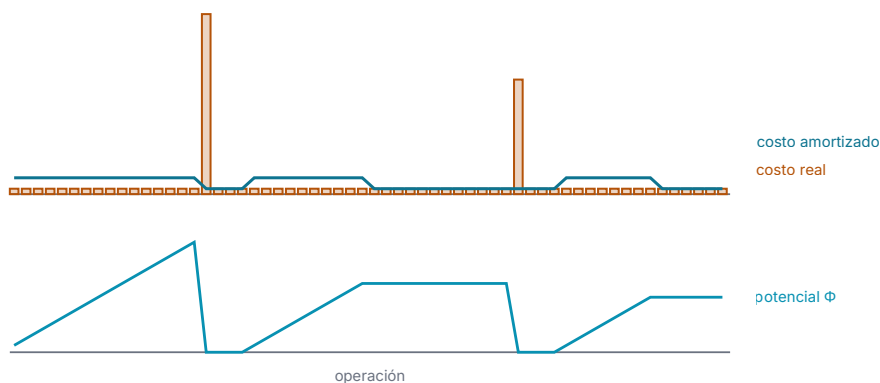


Figura 1. Sesenta operaciones sobre la cola: dieciséis encolar, cuatro desencolar, diez encolar, dieciséis desencolar, ocho encolar, seis desencolar. Los dos picos de costo real son las mudanzas, y en cada una el potencial cae exactamente lo que costó. El costo amortizado no pasa de 3.

La [figura 1](#) es la misma idea en un dibujo. Las barras son el costo real y tienen picos. La línea del medio es el costo amortizado, que no pasa de 3. El potencial de abajo sube despacio mientras se encola y se desploma en cada mudanza, justo lo que la mudanza cuesta.

3. Qué es un buen potencial

Un potencial sirve si cumple tres cosas:

- Empieza en un valor fijo, casi siempre 0.
- Nunca baja de ese valor.
- Sube cuando la estructura acumula una **deuda**: algo que una operación futura va a tener que pagar. Y baja cuando esa operación la paga.

La tercera es la que hay que inventar, y la pregunta que ayuda es: ¿qué trabajo está dejando pendiente esta estructura? En la cola, los elementos que todavía no se mudaron. En el arreglo dinámico, lo cerca que está de llenarse. En un árbol que se reorganiza solo, que viene ahora, los caminos largos que todavía nadie recorrió.

Compara con la conferencia 3. Allí el potencial de Gale-Shapley solo bajaba, una unidad por propuesta, y servía para contar vueltas. Aquí baja en las operaciones caras y sube en las baratas, y el costo amortizado de cada operación es su costo real corregido por ese movimiento. Es la misma herramienta, una cantidad que no puede bajar de un piso, usada para acotar costo en vez de para demostrar que algo termina.

4. Splay trees: el algoritmo

Un árbol binario de búsqueda cuesta lo que mide su profundidad. Los árboles AVL y rojinegros guardan información de balance en cada nodo para que la profundidad sea $O(\log n)$ siempre. Queremos algo más simple: un árbol **sin ninguna información de balance**, que se reorganice según los accesos, y que aun así garantice $O(\log n)$ amortizado por operación.

Cómo se le ocurre a uno. Si un elemento se usa, conviene que quede arriba. La primera idea es la de Brian Allen e Ian Munro (1978): después de acceder a un nodo, **subirlo a la raíz** con rotaciones simples, una por nivel. Es natural, y cada rotación preserva el orden del árbol de búsqueda. Pero no alcanza, y la sección 6 va a medir cuánto no alcanza.

La corrección es de Daniel Sleator y Robert Tarjan (1985), y se llama **splay**. Sube el nodo x a la raíz de dos niveles en dos, mirando la forma del camino:

- **Zig:** el padre y de x es la raíz. Una rotación.
- **Zig-zig:** x y su padre y son hijos del mismo lado. Se rota **primero el padre** y con el abuelo z , y después x con y .
- **Zig-zag:** x y y son hijos de lados distintos. Se rota x dos veces.

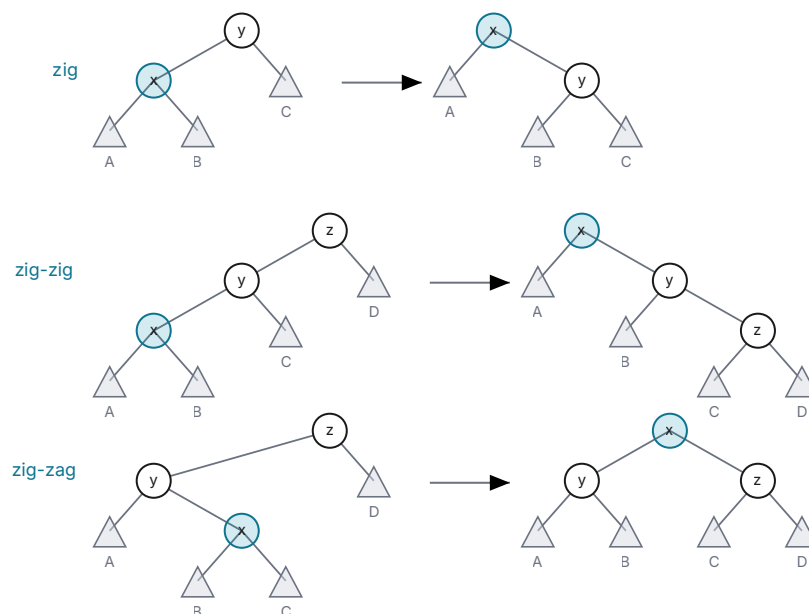


Figura 2. Los tres pasos del splay. Los triángulos son subárboles que no cambian por dentro. En zig-zag las rotaciones son las mismas que haría subir a la raíz; en zig-zig no, porque se rota primero el padre.

La [figura 2](#) muestra los tres pasos. Fíjate en que zig y zig-zag hacen exactamente las mismas rotaciones que subir a la raíz. **La única diferencia entre las dos heurísticas está en el caso zig-zig**, en el orden de dos rotaciones. Toda la sección 5 va a consistir en explicar por qué ese orden importa.

```
class Nodo:
    def __init__(self, clave):
        self.clave, self.izq, self.der, self.padre = clave, None, None, None

def rotar(x):
    """Sube x un nivel rotando la arista con su padre. Preserva el orden."""
    p, g = x.padre, x.padre.padre
    if p.izq is x:
        p.izq, x.der = x.der, p
        if p.izq:
            p.izq.padre = p
    else:
        p.der, x.izq = x.izq, p
        if p.der:
            p.der.padre = p
    p.padre, x.padre = x, g
    if g:
        if g.izq is p:
            g.izq = x
        else:
            g.der = x

def splay(x):
    rotaciones = 0
    while x.padre:
        p, g = x.padre, x.padre.padre
        if g is None:
            rotar(x)
            rotaciones += 1
        elif (g.izq is p) == (p.izq is x):
            rotar(p)
            rotar(x)
            rotaciones += 2
        else:
            rotar(x)
            rotar(x)
            rotaciones += 2
    return rotaciones

def subir_a_la_raiz(x):
    rotaciones = 0
    while x.padre:
        rotar(x)
        rotaciones += 1
    return rotaciones

def buscar(raiz, clave):
    x = raiz
    while x.clave != clave:
        x = x.izq if clave < x.clave else x.der
    return x

def camino(n):
    """El peor árbol inicial: n en la raíz, cada clave hija izquierda de la siguiente."""
    nodos = [Nodo(k) for k in range(1, n + 1)]
    for k in range(1, n):
        nodos[k].izq, nodos[k - 1].padre = nodos[k - 1], nodos[k]
    return nodos[-1]
```

Un acceso busca la clave y después reorganiza. El costo de un acceso es proporcional a la profundidad del nodo, que es el número de rotaciones que hace falta para subirlo, así que contamos rotaciones, y 1 cuando el nodo ya es la raíz.

5. Splay trees: el lema de acceso

Para cada nodo x , sea $s(x)$ el número de nodos de su subárbol (contándose a sí mismo), y $r(x) = \log_2 s(x)$ su **rango**. El potencial del árbol es la suma de los rangos:

$$\Phi = \sum_x r(x).$$

¿Qué deuda mide? Un camino de n nodos tiene rangos $\log_2 1, \log_2 2, \dots, \log_2 n$, que suman $\log_2 n! = \Theta(n \log n)$. Un árbol balanceado tiene potencial $\Theta(n)$. El potencial es alto cuando el árbol es un camino, que es justamente cuando los accesos van a ser caros. Un acceso caro va a tener que bajarlo.

Lema de acceso (Sleator y Tarjan). El costo amortizado de hacer splay de un nodo x en un árbol con raíz t es a lo sumo

$$3(r(t) - r(x)) + 1 = O(\log n).$$

Antes de la demostración, una desigualdad que es el único ingrediente que no es aritmética directa.

Lema de concavidad. Si $a, b > 0$ y $a + b \leq c$, entonces $\log_2 a + \log_2 b \leq 2 \log_2 c - 2$.

Demostración. $ab \leq \left(\frac{a+b}{2}\right)^2 \leq \frac{c^2}{4}$, por la desigualdad entre las medias aritmética y geométrica. Tomando $\log_2$ a los dos lados, $\log_2 a + \log_2 b \leq 2 \log_2 c - 2$. $\square$

Demostración del lema de acceso. Analizamos un paso del splay a la vez. Sea y el padre de x y z su abuelo, y escribamos r para los rangos antes del paso y r' para los de después. Solo cambian los rangos de los nodos que rotan, porque los subárboles del resto no cambian de contenido. Vamos a ver que cada paso tiene costo amortizado a lo sumo $3(r'(x) - r(x))$, más 1 en el caso zig.

Dos hechos se usan en todos los casos. Después del paso, x ocupa el lugar que ocupaba el nodo más alto que rotó, con el mismo subárbol, así que $r'(x)$ es igual al rango anterior de ese nodo. Y como x estaba debajo de y , $r(y) \geq r(x)$.

- **Zig.** Costo real 1. Cambian x e y , y $r'(x) = r(y)$, así que $\Delta\Phi = r'(y) - r(x)$. Como y queda debajo de x , $r'(y) \leq r'(x)$. El costo amortizado es a lo sumo $1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x))$.
- **Zig-zig.** Costo real 2. Cambian x, y, z , y $r'(x) = r(z)$, así que $\Delta\Phi = r'(y) + r'(z) - r(x) - r(y)$. Usando $r'(y) \leq r'(x)$ y $r(y) \geq r(x)$, queda $\Delta\Phi \leq r'(x) + r'(z) - 2r(x)$. Falta ver que $2 + r'(x) + r'(z) - 2r(x) \leq 3(r'(x) - r(x))$, es decir, $r(x) + r'(z) \leq 2r'(x) - 2$. Aquí entra la concavidad: el subárbol viejo de x y el subárbol nuevo de z no comparten nodos (mira la [figura 2](#): el primero tiene A y B , el segundo C y D), y los dos están dentro del subárbol nuevo de x . Así que $s(x) + s'(z) \leq s'(x)$, y el lema de concavidad da justo lo que faltaba.
- **Zig-zag.** Costo real 2. Igual que antes, $\Delta\Phi \leq r'(y) + r'(z) - 2r(x)$. Ahora los subárboles nuevos de y y z no comparten nodos y están dentro del subárbol nuevo de x , así que $s'(y) + s'(z) \leq s'(x)$, y por concavidad $r'(y) + r'(z) \leq 2r'(x) - 2$. El costo amortizado es a lo sumo $2 + 2r'(x) - 2 - 2r(x) \leq 3(r'(x) - r(x))$.

Al sumar sobre todos los pasos, las diferencias $r'(x) - r(x)$ se telescopian: el rango de x empieza en $r(x)$ y termina en $r(t)$, porque al final x es la raíz y su subárbol es el árbol entero. El zig ocurre a lo sumo una vez, en el último paso. Total: $3(r(t) - r(x)) + 1$. $\square$

Consecuencia. Como $r(t) = \log_2 n$ y $r(x) \geq 0$, cada acceso cuesta $O(\log n)$ amortizado. Para m accesos,

$$\sum c_i = \sum \hat{c}_i - (\Phi_m - \Phi_0) \leq m(3 \log_2 n + 1) + \Phi_0,$$

porque $\Phi_m \geq 0$. El potencial inicial es a lo sumo $\log_2 n! \leq n \log_2 n$, el de un camino. En total, $O((m+n) \log n)$. Aquí la condición $\Phi_m \geq \Phi_0$ no se cumple, y la demostración lo resuelve pagando Φ_0 aparte, una sola vez.

Por qué subir a la raíz no alcanza, leído en la demostración. Una rotación simple es un zig en medio del camino, y su costo amortizado es a lo sumo $1 + r'(x) - r(x)$, por la misma cuenta del primer caso. Subir a la raíz desde profundidad d son d rotaciones simples, y la cota que sale es $d + (r(t) - r(x))$: los rangos se telescopian, pero los d unos no. Para bajar de $O(d)$ a $O(\log n)$ hay que pagar el $+1$ de cada rotación, y eso lo hace el -2 de la concavidad, que solo aparece cuando dos rotaciones se agrupan en un paso. En zig-zag, las dos rotaciones de subir a la raíz ya son las del splay y la cuenta cierra. En zig-zig, la cuenta solo cierra si se rota primero el padre, porque solo así el subárbol viejo de x y el nuevo de z quedan separados dentro del nuevo de x .

Un lema como este también se puede ejecutar. El bloque siguiente calcula el potencial antes y después de cada acceso y comprueba la cota, para las dos heurísticas, sobre la misma secuencia de accesos.

```
def tamano_subarbol(x):
    return 0 if x is None else 1 + tamano_subarbol(x.izq) + tamano_subarbol(x.der)

def potencial(x):
    if x is None:
        return 0.0
    return math.log2(tamano_subarbol(x)) + potencial(x.izq) + potencial(x.der)

def vigilar_lemma(reorganizar, n=60, accesos=1500, semilla=4):
    random.seed(semilla)
    raiz, violaciones, peor = camino(n), 0, 0.0
    for i in range(accesos):
        clave = random.randint(1, n) if i % 2 else (i // 2) % n + 1
        x = buscar(raiz, clave)
        antes, r_x = potencial(raiz), math.log2(tamano_subarbol(x))
        rotaciones = reorganizar(x)
        raiz = x
        amortizado = max(rotaciones, 1) + potencial(raiz) - antes
        exceso = amortizado - (3 * (math.log2(n) - r_x) + 1)
        violaciones += exceso > 1e-9
        peor = max(peor, exceso)
    return violaciones, peor

for nombre, f in [("splay", splay), ("subir a la raíz", subir_a_la_raiz)]:
    v, peor = vigilar_lemma(f)
    print(f"{nombre:>16}: la cota falla en {v} de 1500 accesos, "
          f"el peor exceso es {peor:.1f}")
```

```
splay: la cota falla en 0 de 1500 accesos, el peor exceso es 0.0
subir a la raíz: la cota falla en 4 de 1500 accesos, el peor exceso es 40.3
```

El splay no viola la cota nunca, que es lo que dice el lema. Subir a la raíz la viola pocas veces, pero por mucho. No es un defecto de la demostración, que solo dice que la cuenta no cierra; el experimento dice que tampoco cierra el costo. La sección siguiente busca la secuencia que hace que las violaciones se repitan.

6. El error que la demostración anuncia

La demostración dice dónde buscar: en los pasos zig-zig, que aparecen cuando el camino de acceso va siempre hacia el mismo lado. El caso extremo es un árbol camino, y la secuencia que lo explota es acceder a las claves en orden, 1, 2, ..., n , una y otra vez. Con cada heurística medimos el costo por acceso durante tres rondas completas.

```
def costo_por_acceso(reorganizar, n, rondas=3):
    raiz, total = camino(n), 0
    for _ in range(rondas):
        for clave in range(1, n + 1):
            x = buscar(raiz, clave)
            total += max(reorganizar(x), 1)
            raiz = x
    return total / (rondas * n)

medidas = {"splay": [], "subir a la raíz": []}
print(f"{'n':>6} {'splay':>8} {'subir a la raíz':>16} {'log2 n':>8}")
for n in [100, 200, 400, 800]:
    s, r = costo_por_acceso(splay, n), costo_por_acceso(subir_a_la_raiz, n)
    medidas["splay"].append((n, s))
    medidas["subir a la raíz"].append((n, r))
print(f"{'n':>6} {'s':>8.1f} {'r':>16.1f} {'math.log2(n):>8.1f}")
```

n	splay	subir a la raíz	$\log_2 n$
100	4.2	50.5	6.6
200	4.2	100.5	7.6
400	4.3	200.5	8.6
800	4.4	400.5	9.6

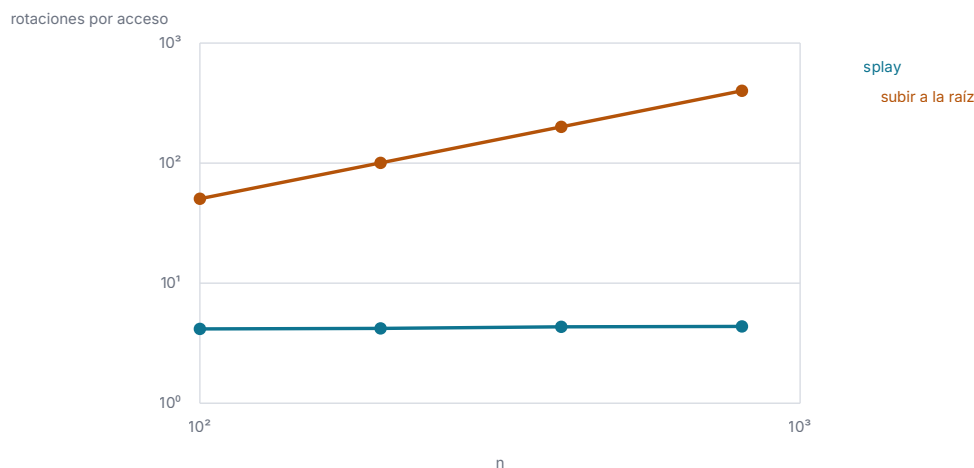


Figura 3. Costo por acceso al recorrer las claves en orden, en ejes logarítmicos. Subir a la raíz tiene pendiente 1: cada acceso cuesta $\Theta(n)$. Splay es plano: cada acceso cuesta una constante.

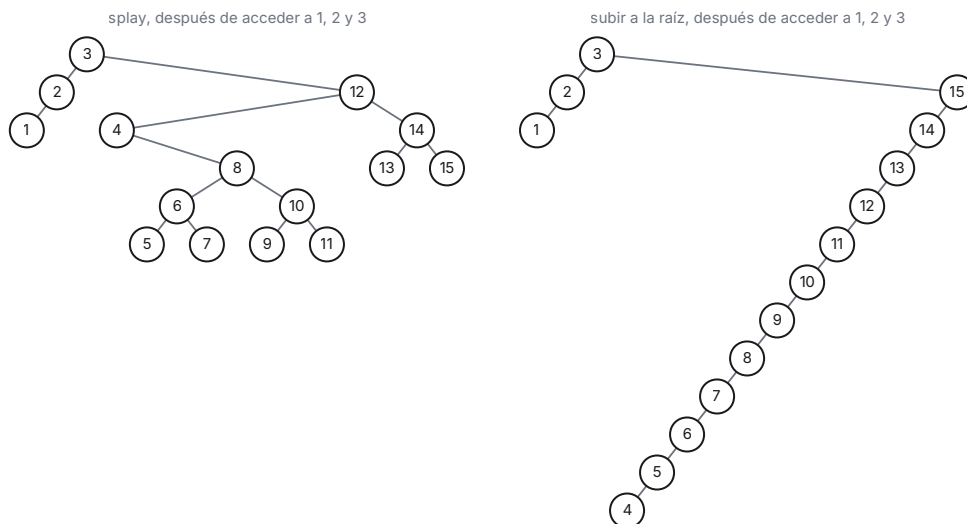


Figura 4. Un camino de quince nodos después de acceder a 1, 2 y 3, con cada heurística. Splay dejó un árbol de profundidad mucho menor. Subir a la raíz dejó otro camino casi tan largo como el original, listo para que el próximo acceso vuelva a costar lo mismo.

La tabla y la [figura 3](#) dicen lo mismo: splay cuesta una constante por acceso sin importar n , y subir a la raíz cuesta $n/2$, que es $\Theta(n)$. La [figura 4](#) muestra por qué. Sleator y Tarjan lo describen así: el splay reduce aproximadamente a la mitad la profundidad de cada nodo del camino de acceso, y subir a la raíz no tiene esa propiedad. Después de cada acceso, subir a la raíz deja un camino casi igual de largo que el anterior, y la siguiente clave está otra vez en el fondo.

Fíjate en qué pruebas pasa la heurística mala. Es correcta: el árbol sigue siendo de búsqueda, el nodo accedido termina en la raíz, y cualquier prueba de correctitud pasa. Sobre la secuencia mezclada de la sección 5, la cota se violó en muy pocos accesos. El error aparece de verdad en secuencias con estructura, y la demostración es la que dice cuál: la que repite el caso zig-zig.

7. Union-find: lo que ya sabían y lo que falta

La estructura de conjuntos disjuntos, que vieron en Estructuras de Datos, mantiene una partición de $\{0, \dots, n-1\}$ con dos operaciones: `find(x)` devuelve un representante del conjunto de x , y `union(a, b)` junta los conjuntos de a y b . Cada conjunto es un árbol con punteros al padre, y el representante es la raíz. Hay dos heurísticas:

- **Unión por rango:** cada raíz guarda un **rango**, una cota superior de su altura. Al unir, la raíz de rango menor cuelga de la de rango mayor. Si empatan, se elige una y su rango sube en 1.
- **Compresión de caminos:** después de un `find`, cada nodo del camino recorrido pasa a colgar directamente de la raíz.

```
class UnionFind:
    def __init__(self, n, por_rango=True, compresion=True):
        self.padre, self.rango = list(range(n)), [0] * n
        self.por_rango, self.compresion = por_rango, compresion
        self.pasos = 0  # punteros recorridos por los find

    def find(self, x):
        raiz = x
        while self.padre[raiz] != raiz:
            raiz = self.padre[raiz]
            self.pasos += 1
        if self.compresion:  # segunda pasada: colgar todo de la raíz
            while self.padre[x] != raiz:
```

```

        self.padre[x], x = raiz, self.padre[x]
    return raiz

def union(self, a, b):
    a, b = self.find(a), self.find(b)
    if a == b:
        return
    if self.por_rango and self.rango[a] < self.rango[b]:
        a, b = b, a
    self.padre[b] = a
    if self.por_rango and self.rango[a] == self.rango[b]:
        self.rango[a] += 1

def profundidad(uf, x):
    d = 0
    while uf.padre[x] != x:
        x, d = uf.padre[x], d + 1
    return d

```

Medimos tres configuraciones, cada una contra la entrada que peor le sienta.

```

n = 1 << 12

# sin heurísticas: una cadena, y find siempre al fondo
uf = UnionFind(n, por_rango=False, compresion=False)
for i in range(n - 1):
    uf.union(i + 1, i)
uf.pasos = 0
for _ in range(1000):
    uf.find(0)
print(f"sin heurísticas, cadena:          {uf.pasos / 1000:7.1f} pasos por find")

# árboles binomiales: la peor entrada para la unión por rango
def binomial(uf, n):
    tam = 1
    while tam < n:
        for i in range(0, n, 2 * tam):
            uf.union(i, i + tam)
        tam *= 2

for comprime in (False, True):
    uf = UnionFind(n, compresion=comprime)
    binomial(uf, n)
    hondo = max(range(n), key=lambda x: profundidad(uf, x))
    d = profundidad(uf, hondo)
    uf.pasos = 0
    for _ in range(1000):
        uf.find(hondo)
    print(f"rango{'', compresión' if comprime else ',          '} binomial:")
        f"    {uf.pasos / 1000:7.1f} pasos por find    (profundidad inicial {d},"
        f"    log2 n = {int(math.log2(n))})")

# las dos heurísticas, operaciones al azar
random.seed(2)
uf = UnionFind(n)
for _ in range(4 * n):
    if random.random() < 0.5:
        uf.union(random.randrange(n), random.randrange(n))
    else:
        uf.find(random.randrange(n))
print(f"rango y compresión, al azar:    {uf.pasos / (4 * n):7.1f} pasos por operación")

```

sin heurísticas, cadena:	4095.0 pasos por find	
rango,	binomial:	12.0 pasos por find (profundidad inicial 12, log ₂ n = 12)

rango, compresión binomial:	1.0 pasos por find	(profundidad inicial 12, $\log_2 n = 12$)
rango y compresión, al azar:	1.2 pasos por operación	

Sin heurísticas, un find cuesta $\Theta(n)$. Con unión por rango, los árboles binomiales llegan a profundidad exactamente $\log_2 n$, y cada find al fondo cuesta eso. Con compresión, el primer find paga el camino y los siguientes cuestan 1.

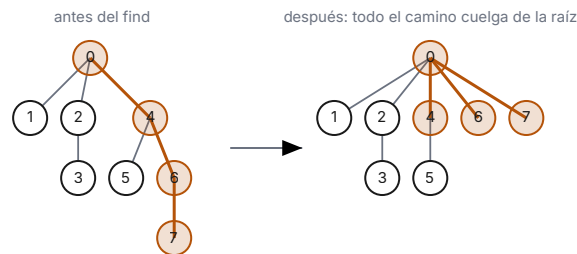


Figura 5. Un árbol binomial de ocho nodos construido con unión por rango, antes y después de find(7). El camino recorrido, en naranja, queda colgando entero de la raíz.

La pregunta de esta conferencia es por qué, con las dos heurísticas juntas, el costo es casi constante siempre, y no solo en las entradas que se nos ocurrió probar.

8. Union-find: la cota $O(m \log^* n)$

Tres propiedades del rango. Las tres son sobre la estructura con unión por rango y compresión, en cualquier momento de cualquier secuencia.

1. Si x no es raíz, $\text{rango}(x) < \text{rango}(\text{padre}(x))$, y el rango de x ya no cambia. Un nodo solo cambia de rango siendo raíz. Cuando cuelga de otra raíz, la unión eligió una de rango mayor, o de rango igual que acto seguido sube. La compresión solo le cambia el padre por un ancestro, que por inducción tiene rango aún mayor.
2. Un nodo que llega a rango r es, en ese momento, raíz de un árbol con al menos 2^r nodos. Por inducción: el rango sube a r solo al unir dos raíces de rango $r - 1$, cada una con al menos 2^{r-1} nodos.
3. Hay a lo sumo $n/2^r$ nodos de rango r . Cuando un nodo alcanza rango r , marquemos con su nombre los al menos 2^r nodos de su árbol. Ningún nodo recibe dos marcas de rango r : si más tarde otro nodo v alcanzara rango r con un nodo ya marcado en su árbol, el primero estaría ahora debajo de v con rango al menos r , y por la propiedad 1 el rango de v sería mayor que r . Contradicción. Así que hay a lo sumo $n/2^r$ nodos de rango r , y en particular ningún rango pasa de $\log_2 n$.

Comprobemos la tercera sobre una ejecución grande.

```
n = 1 << 16
random.seed(6)
uf = UnionFind(n)
for _ in range(3 * n):
    uf.union(random.randrange(n), random.randrange(n))
cuenta = {}
for r in uf.rango:
    cuenta[r] = cuenta.get(r, 0) + 1
print(f"{'rango':>6} {'nodos':>8} {'cota n/2^r':>11}")
for r in sorted(cuenta):
    print(f"{'r':>6} {'cuenta[r]:>8} {n / 2 ** r:>11.0f}")
```

rango	nodos	cota $n/2^r$
0	49212	65536
1	12967	32768
2	2774	16384
3	499	8192
4	73	4096
5	10	2048
6	1	1024

El logaritmo iterado. $\log^* n$ es cuántas veces hay que aplicar $\log_2$ a n hasta bajar a 1 o menos.

```
def log_estrella(n):
    k = 0
    while n > 1:
        n, k = math.log2(n), k + 1
    return k

for texto, valor in [("2", 2), ("4", 4), ("16", 16), ("65 536", 2 ** 16),
                    ("2^65536", 2 ** 65536)]:
    print(f"log* {texto:>8} = {log_estrella(valor)}")
```

```
log*      2 = 1
log*      4 = 2
log*     16 = 3
log*   65 536 = 4
log* 2^65536 = 5
```

El número 2^{65536} tiene casi veinte mil cifras decimales. Para cualquier n que quepa en una computadora, $\log^* n \leq 5$.

Grupos de rango. Partimos los rangos posibles en grupos con fronteras en las torres de dos, 1, 2, 4, 16, 65536, ..., donde cada frontera es 2 elevado a la anterior. Los grupos son $\{0, 1\}$, $\{2\}$, $\{3, 4\}$, $\{5, \dots, 16\}$, $\{17, \dots, 65536\}$, y así. En general, un grupo es un intervalo $(k, 2^k]$ con k una torre. Como los rangos no pasan de $\log_2 n$, hay a lo sumo $\log^* n + 1$ grupos.

El conteo. Un `find(x)` recorre un camino $x = x_0, x_1, \dots, x_\ell$ hasta la raíz. Su costo real es ℓ , uno por cada paso de x_i a su padre x_{i+1} . Repartimos cada paso a uno de dos pagadores:

- **Lo paga la operación** si x_{i+1} es la raíz, o si x_i y x_{i+1} están en grupos distintos. Por la propiedad 1 los rangos crecen estrictamente al subir, así que el grupo cambia a lo sumo una vez por grupo. Cada `find` paga a lo sumo $\log^* n + 2$ pasos.
- **Lo paga el nodo** x_i si x_i y su padre están en el mismo grupo y el padre no es la raíz. Después del `find`, la compresión cuelga a x_i de la raíz, que tiene rango mayor que el de su padre anterior. Cada vez que se le cobra a x_i , el rango de su padre sube al menos 1.

Ahora contamos lo que pagan los nodos. Un nodo x con rango en el grupo $(k, 2^k]$ tiene rango fijo, por la propiedad 1. Cada cobro sube el rango de su padre, y después de a lo sumo 2^k cobros el padre tiene rango mayor que 2^k : está en otro grupo, y desde entonces los pasos desde x los paga la operación. Por la propiedad 3, el grupo $(k, 2^k]$ tiene a lo sumo

$$\sum_{r=k+1}^{2^k} \frac{n}{2^r} < \frac{n}{2^k}$$

nodos, y cada uno paga a lo sumo 2^k : el grupo entero paga menos de n . Sumando sobre los grupos, los nodos pagan a lo sumo $n(\log^* n + 1)$ en toda la ejecución.

Teorema (Hopcroft y Ullman, 1973). Una secuencia de m operaciones sobre n elementos, con unión por rango y compresión de caminos, cuesta $O((m + n) \log^* n)$.

Demostración. Cada union hace dos find y trabajo constante. Los find pagan $O(m \log^* n)$ entre todos, y los nodos $O(n \log^* n)$. $\square$

Este es un potencial aunque no lo parezca: el crédito que cada nodo va acumulando mientras su padre sube dentro de un grupo. Lo que no hay es una fórmula cerrada para Φ . La contabilidad por grupos es lo que en la sección 2 llamamos el método de contabilidad, con el crédito asignado a nodos y no a elementos de una pila.

9. La cota de Tarjan y la cota mínima

La cota $\log^* n$ no es la verdad exacta. Robert Tarjan (1975) demostró que el mismo algoritmo cuesta $\Theta(m \alpha(m, n))$, donde α es una inversa de la función de Ackermann, que crece todavía más despacio. Lo enunciamos sin demostrar.

Para ver cuánto más despacio, la definición de CLRS. Sea $A_0(j) = j + 1$, y $A_k(j)$ el resultado de aplicar A_{k-1} a j , $j + 1$ veces seguidas. Entonces $\alpha(n)$ es el menor k con $A_k(1) \geq n$.

```
def A(k, j):
    if k == 0:
        return j + 1
    for _ in range(j + 1):
        j = A(k - 1, j)
    return j

print([A(k, 1) for k in range(4)])
```

[2, 3, 7, 2047]

$A_3(1) = 2047$, y $A_4(1) = A_3(A_3(1)) = A_3(2047)$, que es mayor que $A_2(2047) = 2^{2048} \cdot 2048 - 1$. Así que $\alpha(n) \leq 4$ para todo $n \leq 2^{2048}$, un número de más de seiscientas cifras.

¿Se puede hacer mejor con otra estructura? No. Michael Fredman y Michael Saks (1989) demostraron que cualquier estructura para este problema necesita $\Omega(\alpha(m, n))$ por operación, amortizado, en el modelo de sondeo de celdas: un modelo que solo cuenta cuántas palabras de memoria se leen o escriben, y deja gratis todo el cómputo. La cota mínima vale incluso con ese regalo. Es la conferencia 5 la que se ocupa de cómo se demuestran cosas así.

Para cualquier entrada real, α es a lo sumo 4, y union-find es constante en la práctica. Pero la diferencia entre "a lo sumo 4 en la práctica" y "constante" es un teorema: no existe una estructura con $O(1)$ amortizado para este problema.

10. Cierre

Cada estructura tuvo su potencial, y cada potencial medía una deuda:

ESTRUCTURA	POTENCIAL	QUÉ DEUDA MIDE
cola con dos pilas	2 · tamaño de la entrada	elementos que todavía no se mudaron

ESTRUCTURA	POTENCIAL	QUÉ DEUDA MIDE
splay tree	$\sum_x \log_2 s(x)$	caminos largos que todavía no se recorrieron
union-find	crédito por grupos de rango	nodos cuyo padre todavía puede subir dentro del grupo

Y cada demostración tuvo su paso delicado. En la cola, exigir $\Phi_m \geq \Phi_0$. En el splay, la concavidad que paga el +1 de cada rotación, y que solo aparece si el zig-zig rota primero el padre. En union-find, la propiedad 3, que acota cuántos nodos hay en cada grupo.

Lo que hay que llevarse es qué garantiza el costo amortizado y qué no. Garantiza el costo total de cualquier secuencia. No garantiza nada sobre una operación suelta: un desenterrar puede costar $\Theta(n)$, y un acceso a un splay tree también. Si una aplicación necesita que **cada** operación sea rápida (un sistema de tiempo real, una interfaz que no puede congelarse), el costo amortizado no sirve, y el ejercicio 8 pide un ejemplo. En la conferencia 21 aparece otra clase de garantía, la esperada, que sí tiene probabilidad. Es otra cosa, y conviene no confundirlas.

11. Resumen

- El costo amortizado es una cota de peor caso sobre secuencias de operaciones. No es un caso promedio y no tiene probabilidad.
- Hay tres métodos: agregado (acotar el total), contabilidad (cobrar de más y guardar crédito) y potencial (una función del estado). El potencial es la contabilidad sin nombres.
- Con el potencial, $\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$, y la suma funciona solo si $\Phi_m \geq \Phi_0$.
- Un buen potencial mide una deuda: el trabajo que la estructura deja pendiente.
- Splay cuesta $O(\log n)$ amortizado por el lema de acceso, con potencial $\sum \log_2 s(x)$. La concavidad del logaritmo paga las rotaciones de a dos.
- Subir a la raíz con rotaciones simples es correcto pero cuesta $\Theta(n)$ por acceso en secuencias ordenadas. La diferencia con splay es el orden de dos rotaciones en el caso zig-zig, y la demostración dice por qué.
- Union-find con rango y compresión cuesta $O((m+n) \log^* n)$, por un conteo por grupos de rango que es un potencial sin fórmula.
- La cota exacta es $\Theta(m \alpha(m, n))$ (Tarjan), y ninguna estructura puede hacerlo mejor (Fredman y Saks). $\alpha(n) \leq 4$ para toda entrada real.

Ejercicios

1. Un arreglo dinámico que duplica su capacidad al llenarse y la reduce a la mitad cuando queda a un cuarto. Encuentra un potencial y demuestra $O(1)$ amortizado. Luego explica por qué reducir a la mitad cuando queda **a la mitad** no funciona: da una secuencia que cueste $\Theta(n)$ por operación.
2. Un contador binario de k bits con incrementar y decrementar. Muestra una secuencia que cueste $\Theta(k)$ por operación. ¿Qué cambia si solo se incrementa?
3. Implementa una cola doble (con inserción y extracción por los dos extremos) usando dos pilas, y analízala. ¿Qué hay que cambiar en la mudanza para que siga siendo $O(1)$ amortizado?
4. Demuestra que, empezando desde un camino de n nodos, acceder a las claves $1, \dots, n$ en orden cuesta $\Theta(n^2)$ en total con subir a la raíz.
5. Union-find con unión por rango, pero olvidando incrementar el rango cuando los dos rangos empatan. ¿Qué profundidad pueden alcanzar los árboles? Si además hay compresión, ¿cuál de las tres propiedades del rango de la sección 8 deja de valer, y qué parte de la demostración cae con ella?

6. Demuestra que la **unión por tamaño** (colgar el árbol con menos nodos del que tiene más) también garantiza profundidad $O(\log n)$ sin compresión.
7. Demuestra que $A_2(j) = 2^{j+1}(j+1) - 1$ y calcula $\log^* n$ y $\alpha(n)$ para $n = 2^{16}$ y $n = 2^{65536}$.
8. Da una aplicación donde una estructura con $O(1)$ amortizado por operación no sirve y hace falta $O(1)$ en el peor caso por operación. Explica qué se rompe.